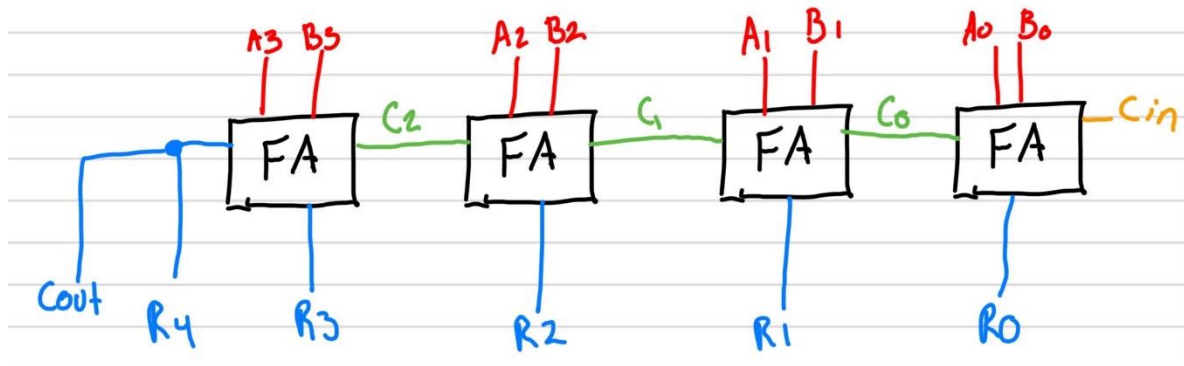


Practice #3: Estructural description in Verilog with Cadence

Reuse the Full-Adder as module to make a ripple carry adder of 4bits+4bits using a carry input and a carry output

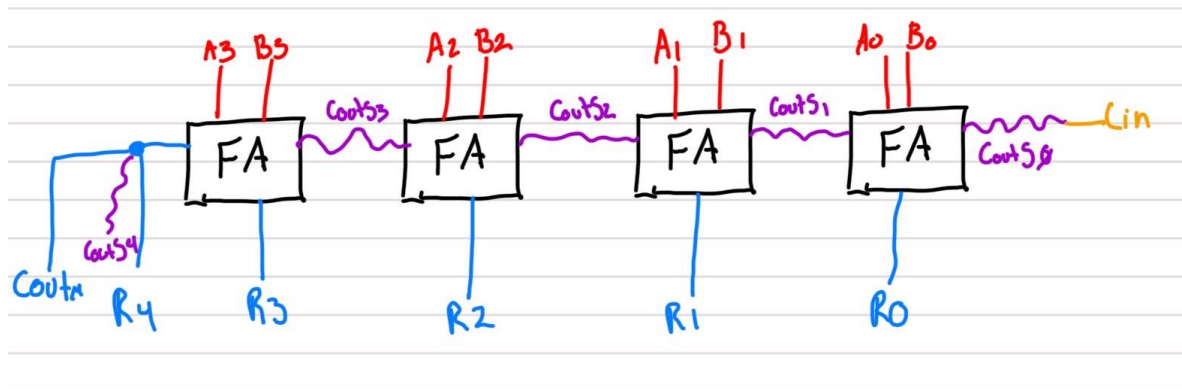
HDL codes

--Using estructures



```
1 //Verilog HDL for "Verilog", "Full_Adder_4b" "functional"
2
3 module Full_Adder_4b (AM,BM,CinM,Result,CoutM);
4
5 input [3:0] AM,BM;
6 input CinM;
7 output [4:0] Result;
8 output CoutM;
9 wire [2:0] CoutS;
10
11 FA U0(.A(AM[0]),.B(BM[0]),.Cin(CinM),.S(Result[0]),.Cout(CoutS[0]));
12 FA U1(.A(AM[1]),.B(BM[1]),.Cin(CoutS[0]),.S(Result[1]),.Cout(CoutS[1]));
13 FA U2(.A(AM[2]),.B(BM[2]),.Cin(CoutS[1]),.S(Result[2]),.Cout(CoutS[2]));
14 FA U3(.A(AM[3]),.B(BM[3]),.Cin(CoutS[2]),.S(Result[3]),.Cout(Result[4]));
15
16 assign CoutM = Result[4];
17
18 endmodule
19
20 //Full Adder Module
21 module FA (A,B,Cin,S,Cout);
22
23 input A,B,Cin;
24 output S,Cout;
25
26 assign S = ((A ^ B) ^ Cin);
27 assign Cout = (((A ^ B) & Cin) | (A & B));
28
29 endmodule
30
```

--Ussing generate



```

1 //Verilog HDL for "Verilog", "Full_Adder_generate" "functional"
2
3
4 module Full_Adder_generate #(parameter N=4) (AM,BM,CinM,Result,CoutM);
5 //Parameter is the amount of adder we will have, the amount of components we will replicate
6
7     input [N-1:0] AM,BM; // we only want 4 bits so N=3 => 0 1 2 3
8     input CinM;
9     output [N:0] Result; //the result its 5 bits => (4 bits for the result and 1bit of the carry out)
10    output CoutM;
11    wire [N:0] CoutS;
12
13    genvar i; //Intern variable (for cicyle)
14
15    assign CoutS[0] = CinM; //we assign the input carry to the signal 'cause its a especial case
16
17    generate
18        for (i=0; i<N; i=i+1) begin
19            FA U0(.A(AM[i]),.B(BM[i]),.Cin(CoutS[i]),.S(Result[i]),.Cout(CoutS[i+1]));
20            //In carry out we +1 to send the carry to the next stage
21        end
22    endgenerate
23
24    assign Result[N] = CoutS[N]; //assign the signal to the result to save the carry out
25
26 endmodule
27
28 //Full Adder Module
29 module FA (A,B,Cin,S,Cout);
30
31     input A,B,Cin;
32     output S,Cout;
33
34     assign S = ((A ^ B) ^ Cin);
35     assign Cout = (((A ^ B) & Cin) | (A & B));
36
37 endmodule
38

```

Test bench codes

--Estructural test bench

```
1 //Verilog HDL for "Verilog", "Full_Adder_4b_tb" "functional"
2
3 module Full_Adder_4b_tb ();
4
5 reg [3:0] AM,BM;
6 reg CinM;
7 wire[4:0] Result;
8 wire CoutM;
9 wire[2:0] CoutS;
10
11 integer i;
12
13 Full_Adder_4b uut(AM,BM,CinM,Result,CoutM);
14
15 initial begin
16
17     for (i=0; i<=15; i=i+1) begin
18
19         AM = $random%16;
20         BM = $random%16;
21         CinM = $random%2;
22
23         #10;
24
25         $display("Iteration => %d ",i);
26         $display("%d+%d+%d= %d ",AM,BM,CinM,Result);
27
28         if(CoutM == 1)begin
29             $display("Carry flag = 1");
30         end
31     end
32
33     $display("Test completed");
34     $finish();
35 end
36
37 endmodule
```

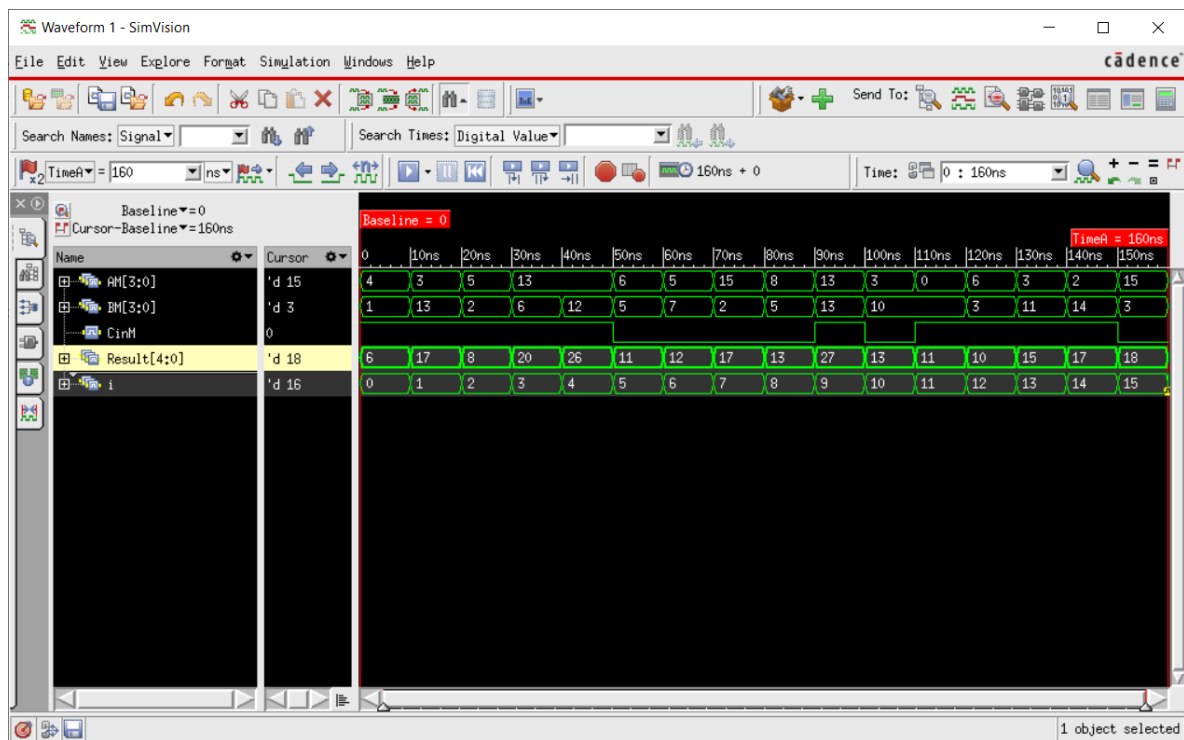
-- Generate test bench

```

1 //Verilog HDL for "Verilog", "Full_Adder_generate_tb" "functional"
2
3 module Full_Adder_generate_tb ();
4
5 localparam N = 4 ; //in the TB we use localparam and not parameter
6 reg [N-1:0] AM,BM;
7 reg CinM;
8 wire [N:0] Result;
9 wire CoutM;
10 wire [N:0] CoutS;
11
12 integer i; //for cycle variable
13
14 Full_Adder_generate uut(AM,BM,CinM,Result,CoutM);
15
16 initial begin
17
18     for (i=0; i<=15; i=i+1) begin
19         //urandom creates a random number
20         AM = $urandom%16; // we write 16 'cause chooses a random number from 0 to 15
21         BM = $urandom%16;
22         CinM = $urandom%2; //we write 2 'cause we choose a random number from 0 to 1
23
24         #10; // waits 10ns
25
26         $display("Iteration => %d ",i);
27         $display("%d+%d+%d= %d ",AM,BM,CinM,Result);
28
29         if(CoutM == 1)begin
30             $display("Carry flag = 1");
31         end
32
33     end
34     $finish();
35     $display("Test completed");
36
37 end
38
39 endmodule

```

--Estructural simulation

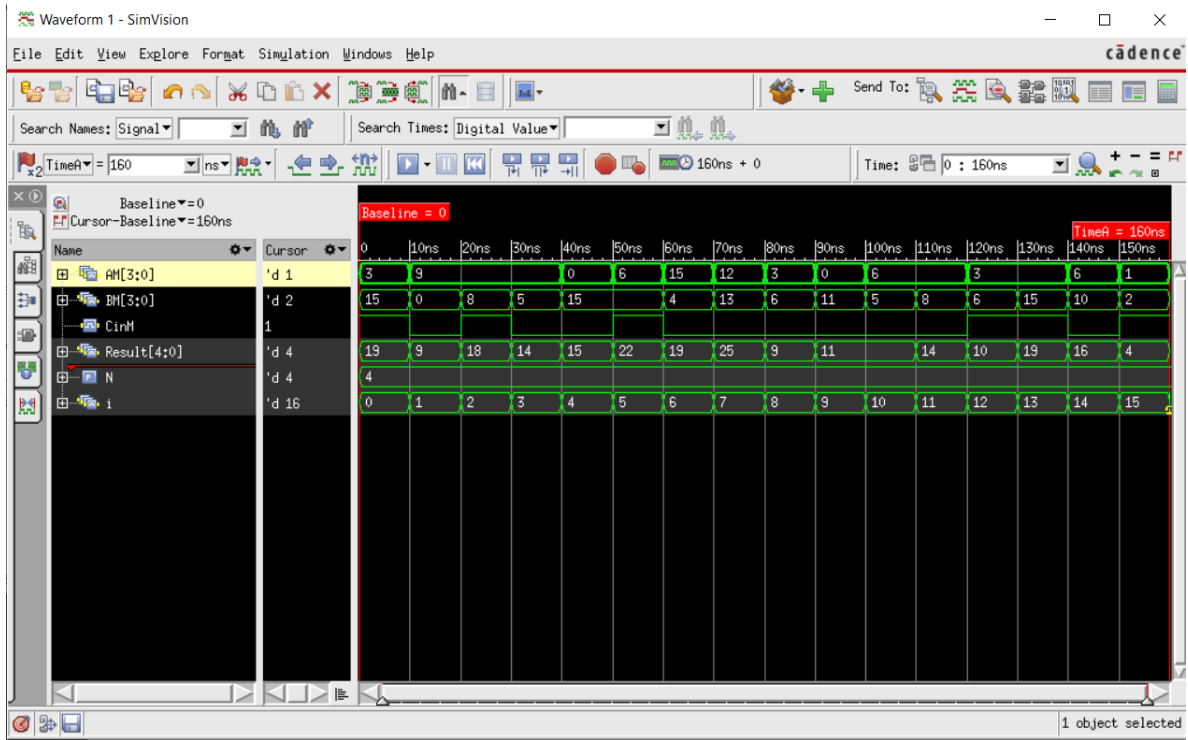


 Composer NC-Verilog Integration Log

File Edit View Help

```
Created default SHM database shmWa
xcelium> probe -create -shm test
xmsim: *E,PRNONE: No new objects t
xcelium> probe -create -shm test.t
Created probe 1
xcelium> run
Iteration => 0
  4+ 1+1= 6
Iteration => 1
  3+13+1= 17
Carry flag = 1
Iteration => 2
  5+ 2+1= 8
Iteration => 3
 13+ 6+1= 20
Carry flag = 1
Iteration => 4
 13+12+1= 26
Carry flag = 1
Iteration => 5
  6+ 5+0= 11
Iteration => 6
  5+ 7+0= 12
Iteration => 7
 15+ 2+0= 17
Carry flag = 1
Iteration => 8
  8+ 5+0= 13
Iteration => 9
 13+13+1= 27
Carry flag = 1
Iteration => 10
  3+10+0= 13
Iteration => 11
  0+10+1= 11
Iteration => 12
  6+ 3+1= 10
Iteration => 13
  3+11+1= 15
Iteration => 14
  2+14+1= 17
Carry flag = 1
Iteration => 15
 15+ 3+0= 18
Carry flag = 1
Test completed
```

--Generate simulation



Composer NC-Verilog Integration

File Edit View Help

```
xcelium> probe -create -shm te
xmsim: *E,PRNONE: No new objec
xcelium> probe -create -shm te
Created probe 1
xcelium> run
Iteration => 0
  3+15+1= 19
Iteration => 1
  9+ 0+0= 9
Iteration => 2
  9+ 8+1= 18
Iteration => 3
  9+ 5+0= 14
Iteration => 4
  0+15+0= 15
Iteration => 5
  6+15+1= 22
Iteration => 6
 15+ 4+0= 19
Iteration => 7
 12+13+0= 25
Iteration => 8
  3+ 6+0= 9
Iteration => 9
  0+11+0= 11
Iteration => 10
  6+ 5+0= 11
Iteration => 11
  6+ 8+0= 14
Iteration => 12
  3+ 6+1= 10
Iteration => 13
  3+15+1= 19
Iteration => 14
  6+10+0= 16
Iteration => 15
  1+ 2+1= 4
```

CODES

```
module Full_Adder_4b (AM,BM,CinM,Result,CoutM);

input [3:0] AM,BM;
input CinM;
output [4:0] Result;
output CoutM;
wire [2:0] CoutS;

FA U0(.A(AM[0]),.B(BM[0]),.Cin(CinM) ,.S(Result[0]),.Cout(CoutS[0]));
FA U1(.A(AM[1]),.B(BM[1]),.Cin(CoutS[0]),.S(Result[1]),.Cout(CoutS[1]));
FA U2(.A(AM[2]),.B(BM[2]),.Cin(CoutS[1]),.S(Result[2]),.Cout(CoutS[2]));
FA U3(.A(AM[3]),.B(BM[3]),.Cin(CoutS[2]),.S(Result[3]),.Cout(Result[4]));

assign CoutM = Result[4];

endmodule

//Full Adder Module
module FA (A,B,Cin,S,Cout);

input A,B,Cin;
output S,Cout;

assign S = ((A ^ B) ^ Cin);
assign Cout = (((A ^ B) & Cin) | (A & B));

endmodule

////////////////////////////////////

//FULL ADDER 4b+4b TestBench
`timescale 1ns/1ns
`include "FAdder4b.v" //Importamos el archivo al cual le hacemos su testbench

module FAdder4b_tb ();

reg [3:0] AM,BM;
reg CinM;
wire[4:0] Result;
wire CoutM;
wire[2:0] CoutS;

integer i;
```

```
Full_Adder_4b uut(AM,BM,CinM,Result,CoutM);
```

```
initial begin
```

```
    for (i=0; i<=15; i=i+1) begin
```

```
        AM = $random%16;
```

```
        BM = $random%16;
```

```
        CinM = $random%2;
```

```
        #10;
```

```
        $display("Iteration => %d ",i);
```

```
        $display("%d+%d+%d= %d ",AM,BM,CinM,Result);
```

```
        if(CoutM == 1)begin
```

```
            $display("Carry flag = 1");
```

```
        end
```

```
    end
```

```
    $display("Test completed");
```

```
    $finish();
```

```
end
```

```
endmodule
```

```
////////////////////////////////
```

```
module Full_Adder_generate #(parameter N=4) (AM,BM,CinM,Result,CoutM);
```

```
    input [N-1:0] AM,BM;
```

```
    input CinM;
```

```
    output [N:0] Result;
```

```
    output CoutM;
```

```
    wire [N:0] CoutS;
```

```
    genvar i; //Intern variable
```

```
    assign CoutS[0] = CinM;
```

```
    generate
```

```
        for (i=0; i<N; i=i+1) begin
```

```
            FA U0(.A(AM[i]),.B(BM[i]),.Cin(CoutS[i]),.S(Result[i]),.Cout(CoutS[i+1]));
```

```

        end
    endgenerate

    assign Result[N] = CoutS[N];

endmodule

//Full Adder Module
module FA (A,B,Cin,S,Cout);

    input A,B,Cin;
    output S,Cout;

    assign S = ((A ^ B) ^ Cin);
    assign Cout = (((A ^ B) & Cin) | (A & B));

endmodule

////////////////////////////////////
//FULL ADDER Nb+Nb TestBench
`timescale 1ns/1ns
`include "FAdder_generate.v" //Importamos el archivo al cual le hacemos su testbench

module FAdder_generate_tb ();

    localparam N = 4 ; //in the TB we use localparam and not parameter
    reg [N-1:0] AM,BM;
    reg CinM;
    wire [N:0] Result;
    wire CoutM;
    wire [N:0] CoutS;

    integer i; //for cycle variable

    Full_Adder_generate uut(AM,BM,CinM,Result,CoutM);

    initial begin

        $dumpfile("FAdder_generate_tb.vcd"); //Creamos el archivo que nos dara la señal de onda
        de nuestra salida
        $dumpvars(0,FAdder_generate_tb);

        for (i=0; i<=15; i=i+1) begin
            //urandom creates a random number

```

```
AM = $urandom%16; // we write 16 'cause chooses a random numbre from 0 to 15
BM = $urandom%16;
CinM = $urandom%2; //we write 2 'cause we choose a random number from 0 to 1

#10; // waits 10ns

$display("Iteration => %d ",i);
$display("%d+%d+%d= %d ",AM,BM,CinM,Result);

if(CoutM == 1)begin
    $display("Carry flag = 1");
end

end
$finish();
$display("Test completed");

end

endmodule
```